

Data Science Lab Manual

Practical 1

Aim: House rent prediction using linear regression.

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

# Create Dataset
data = {
    'Area': [1200, 1500, 1800, None, 2000, 1700, 1600, None],
    'Bedrooms': [2, 3, 3, 2, None, 3, 2, 1],
    'Age': [5, 2, 4, 6, 3, None, 7, 10],
    'Rent': [15000, 22000, 25000, 18000, 27000, 24000, 20000, 12000]
}

df = pd.DataFrame(data)
print("Original Dataset:\n")
print(df)

# Handle Null Values
# Fill numerical null values using mean
df['Area'] = df['Area'].fillna(df['Area'].mean())
df['Bedrooms'] = df['Bedrooms'].fillna(df['Bedrooms'].mean())
df['Age'] = df['Age'].fillna(df['Age'].mean())

print("\nDataset After Handling Null Values:\n")
print(df)
```

```
# Define Features & Target
```

```
X = df[['Area', 'Bedrooms', 'Age']]
```

```
y = df['Rent']
```

```
# Split Dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Train Linear Regression Model
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
# Prediction
```

```
y_pred = model.predict(X_test)
```

```
print("\nPredicted Rent Values:\n")
```

```
print(y_pred)
```

```
#Accuracy Check
```

```
mae = mean_absolute_error(y_test, y_pred)
```

```
print("\nMean Absolute Error:", mae)
```

```
# Predict New House Rent
```

```
new_house = [[1750, 3, 4]] # Area, Bedrooms, Age
```

```
predicted_rent = model.predict(new_house)
```

```
print("\nPredicted Rent for New House:", predicted_rent[0])
```

Practical 2

Aim: Medical diagnosis for disease spread pattern.

Code:

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_absolute_error, r2_score

# Load Dataset
diabetes = load_diabetes()
X = diabetes.data
y = diabetes.target
print("Total Samples:", len(X))

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create Models
models = {
    "Linear Regression": LinearRegression(),
    "Decision Tree": DecisionTreeRegressor(),
    "Random Forest": RandomForestRegressor(),
    "KNN": KNeighborsRegressor()
}
```

```
# Train and Test Models

for name, model in models.items():

    model.fit(X_train, y_train)    # Train Model

    y_pred = model.predict(X_test)  #Prediction

    mae = mean_absolute_error(y_test, y_pred)    # Evaluation

    r2 = r2_score(y_test, y_pred)

    print("Model:", name)

    print("Mean Absolute Error:", mae)

    print("R2 Score:", r2)


#Predict New Patient Data

sample_patient = [X_test[0]]

print("\n\nPrediction for One Patient:\n")

for name, model in models.items():

    prediction = model.predict(sample_patient)

    print(name, "Prediction:", prediction[0])
```

Practical 3

Aim: Automate email classification and response.

Code:

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score


# Load Dataset

url = "https://raw.githubusercontent.com/justmarkham/pycon-2016-tutorial/master/data/sms.tsv"

df = pd.read_csv(url, sep='\t', header=None, names=['Category', 'Message'])
df.head()


# Features and Labels

X = df['Message']
y = df['Category']


# Split Dataset

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


# Convert Text into Numbers

vectorizer = CountVectorizer()

X_train_vector = vectorizer.fit_transform(X_train)
X_test_vector = vectorizer.transform(X_test)


model = MultinomialNB()    # Train Mode
```

```
model.fit(X_train_vector, y_train)

# Prediction

y_pred = model.predict(X_test_vector)


# Accuracy

accuracy = accuracy_score(y_test, y_pred)

print("\nModel Accuracy:", accuracy)


# Automatic Email Classification

new_email = ["Congratulations! You won a free iPhone. Click now!"]

new_email_vector = vectorizer.transform(new_email)

prediction = model.predict(new_email_vector)

print("\nEmail Category:", prediction[0])


# Automatic Response Generation

if prediction[0] == "spam":

    response = "Warning: This email looks suspicious and may be spam."

else:

    response = "This email is safe and important."


print("\nAutomatic Response:")

print(response)
```

Practical 4

Aim: Customer segmentation in business model based on their demographic, psychographic and behavior data.

Code:

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans

from sklearn.preprocessing import LabelEncoder


# Step 1: Load Dataset

url = "https://raw.githubusercontent.com/tirthajyoti/Machine-Learning-with-Python/master/Datasets/Mall_Customers.csv"


df = pd.read_csv(url)

df.head()


# Step 2: Convert Categorical Data

label = LabelEncoder()

df['Gender'] = label.fit_transform(df['Gender'])


# Step 3: Select Features

X = df[['Gender',
        'Age',
        'Annual Income (k$)',
        'Spending Score (1-100)']]


# Step 4: Find Best Number of Clusters

wcss = []
```

```
for i in range(1, 11):  
    kmeans = KMeans(n_clusters=i, random_state=42)  
    kmeans.fit(X)  
    wcss.append(kmeans.inertia_)
```

```
# Plot Elbow Graph  
plt.plot(range(1, 11), wcss, marker='o')  
plt.title("Elbow Method")  
plt.xlabel("Number of Clusters")  
plt.ylabel("WCSS")  
plt.show()
```

```
# Step 6: Apply K-Means  
kmeans = KMeans(n_clusters=5, random_state=42)  
df['Cluster'] = kmeans.fit_predict(X)
```

```
# Step 7: Display Customer Segments  
print("\nCustomer Segments:\n")  
print(df[['CustomerID',  
          'Gender',  
          'Age',  
          'Annual Income (k$)',  
          'Spending Score (1-100)',  
          'Cluster']].head(20))
```

```
# Step 8: Visualize Clusters  
plt.figure(figsize=(8,6))
```

```
plt.scatter(df['Annual Income (k$)'], df['Spending Score (1-100)'],c=df['Cluster'])  
plt.xlabel("Annual Income")  
plt.ylabel("Spending Score")  
plt.title("Customer Segmentation")  
plt.show()
```

```
# Step 9: Predict New Customer Segment
```

```
new_customer = [[1, 30, 70, 80]]
```

```
# Gender=1(Male), Age=30,
```

```
# Income=70k, Spending Score=80
```

```
segment = kmeans.predict(new_customer)
```

```
print("\nNew Customer belongs to Cluster:", segment[0])
```

Practical 5

Aim: Construct a recommendation system based on the customer transaction using Association rule mining.

Code:

```
import pandas as pd

from mlxtend.frequent_patterns import apriori, association_rules

# Step 1: Load Dataset

# Download Dataset:

# https://archive.ics.uci.edu/ml/datasets/online+retail

df = pd.read_excel("Online Retail.xlsx")
df.head()


# Step 2: Remove Null Values

df = df.dropna()


# Step 3: Keep Only Required Columns

df = df[['InvoiceNo', 'Description']]


# Step 4: Create Basket Dataset

basket = pd.crosstab(df['InvoiceNo'], df['Description'])


# Step 5: Convert Values into 0 and 1

basket[basket > 0] = 1


# Step 6: Apply Apriori Algorithm

frequent_items = apriori(basket, min_support=0.02, use_colnames=True)

print("\nFrequent Itemsets:\n")

print(frequent_items.head())
```

Step 7: Generate Association Rules

```
rules = association_rules(  
    frequent_items,  
    metric="confidence",  
    min_threshold=0.3  
)  
print("\nAssociation Rules:\n")  
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']].head())
```

Step 8: Simple Product Recommendation

```
product = "WHITE HANGING HEART T-LIGHT HOLDER"  
print("\nRecommended Products for:", product)
```

```
for i in range(len(rules)):  
    antecedent = list(rules.iloc[i]['antecedents'])  
    if product in antecedent:  
        consequent = list(rules.iloc[i]['consequents'])  
        print("Recommended:", consequent)
```

Practical 6

Aim: Construct a recommendation system based on the customer transaction using Association rule mining.

Code:

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# Step 1: Load Dataset
# Download Dataset:
# https://archive.ics.uci.edu/ml/datasets/Online+Shoppers+Purchasing+Intention+Dataset

df = pd.read_csv("online_shoppers_intention.csv")
df.head()

# Step 2: Convert Categorical Data
label = LabelEncoder()
categorical_columns = [ 'Month', 'VisitorType', 'Weekend']

for col in categorical_columns:
    df[col] = label.fit_transform(df[col])

# Convert Target Column
df['Revenue'] = label.fit_transform(df['Revenue'])

# Step 3: Features and Target
X = df.drop('Revenue', axis=1)

y = df['Revenue']
```

Step 4: Split Dataset

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Step 5: Train Model

```
model = RandomForestClassifier()
```

```
model.fit(X_train, y_train)
```

Step 6: Prediction

```
y_pred = model.predict(X_test)
```

Step 7: Accuracy

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("\nModel Accuracy:", accuracy)
```

```
print("\nClassification Report:\n")
```

```
print(classification_report(y_test, y_pred))
```

Step 8: Predict Customer Purchase Behavior

```
sample_customer = [X_test.iloc[0]]
```

```
prediction = model.predict(sample_customer)
```

```
if prediction[0] == 1:
```

```
    print("\nCustomer is likely to PURCHASE product")
```

```
else:
```

```
    print("\nCustomer is NOT likely to purchase product")
```

Step 9: Feature Importance

```
importance = pd.DataFrame({
```

```
    'Feature': X.columns,
```

```
    'Importance': model.feature_importances_
```

})

```
importance = importance.sort_values(  
    by='Importance',  
    ascending=False  
)
```

```
print("\nImportant Behavioral Factors:\n")  
print(importance.head(10))
```

Practical 7

Aim: Agricultural data analysis for yield prediction and crop selection on Indian terrain data set.

Code:

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import mean_absolute_error, accuracy_score


# Step 1: Load Dataset
# Download Dataset:
# https://www.kaggle.com/datasets/atharvaingle/crop-recommendation-dataset
df = pd.read_csv("Crop_recommendation.csv")
df.head()


# Step 2: Features and Target
# Features for Indian terrain analysis
X = df[['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']]

# Target for Crop Selection
y_crop = df['label']


# Step 3: Convert Crop Labels into Numbers
label_encoder = LabelEncoder()
y_crop_encoded = label_encoder.fit_transform(y_crop)


# Step 4: Train-Test Split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y_crop_encoded, test_size=0.2,  
random_state=42)
```

```
# Step 5: Train Crop Selection Model
```

```
crop_model = DecisionTreeClassifier()
```

```
crop_model.fit(X_train, y_train)
```

```
# Step 6: Predict Crop
```

```
crop_prediction = crop_model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, crop_prediction)
```

```
print("\nCrop Selection Model Accuracy:", accuracy)
```

```
# Step 7: Yield Prediction Dataset
```

```
# Create simple yield column for demonstration
```

```
# (Estimated agricultural yield)
```

```
df['yield'] = (  
    df['N'] * 0.3 +  
    df['P'] * 0.2 +  
    df['K'] * 0.4 +  
    df['rainfall'] * 0.1  
)
```

```
# Step 8: Yield Prediction Model
```

```
X_yield = X
```

```
y_yield = df['yield']
```

```
X_train_y, X_test_y, y_train_y, y_test_y = train_test_split(  
    X_yield,  
    y_yield,  
    test_size=0.2,  
    random_state=42
```

)

```
yield_model = LinearRegression()  
yield_model.fit(X_train_y, y_train_y)
```

```
# Step 9: Predict Yield
```

```
yield_prediction = yield_model.predict(X_test_y)
```

```
mae = mean_absolute_error(y_test_y, yield_prediction)  
print("\nYield Prediction MAE:", mae)
```

```
# Step 10: Predict New Indian Terrain Data
```

```
# Example Input:N, P, K, temperature, humidity, ph, rainfall
```

```
new_data = [[90, 42, 43, 21, 82, 6.5, 202]]
```

```
# Crop Prediction
```

```
predicted_crop = crop_model.predict(new_data)  
crop_name = label_encoder.inverse_transform(predicted_crop)  
print("\nRecommended Crop:", crop_name[0])
```

```
# Yield Prediction
```

```
predicted_yield = yield_model.predict(new_data)  
print("Estimated Yield:", predicted_yield[0])
```

Practical 8

Aim: Agricultural data analysis for yield prediction and crop selection on Indian terrain data set.

Code:

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, r2_score

# Step 1: Load Dataset
# Download Dataset:
# https://www.kaggle.com/datasets/sudalairajkumar/indian-startup-funding

df = pd.read_csv("startup_funding.csv")
df.head()

# Step 2: Select Important Columns
df = df[[
    'Date dd/mm/yyyy',
    'Industry Vertical',
    'City Location',
    'InvestmentnType',
    'Amount in USD'
]]

# Step 3: Handle Missing Values
df = df.dropna()
```

```
# Step 4: Convert Funding Amount
df['Amount in USD'] = df['Amount in USD'].astype(str)
df['Amount in USD'] = df['Amount in USD'].str.replace(',', '')
df = df[df['Amount in USD'].str.isnumeric()]
df['Amount in USD'] = df['Amount in USD'].astype(float)

# Step 5: Convert Date into Year
df['Year'] = pd.to_datetime(
    df['Date dd/mm/yyyy'],
    errors='coerce'
).dt.year

df = df.dropna()

# Step 6: Encode Categorical Data
label = LabelEncoder()
df['Industry Vertical'] = label.fit_transform(df['Industry Vertical'])
df['City Location'] = label.fit_transform(df['City Location'])
df['InvestmentnType'] = label.fit_transform(df['InvestmentnType'])

# Step 7: Features and Target
X = df[[
    'Year',
    'Industry Vertical',
    'City Location',
    'InvestmentnType'
]]
y = df['Amount in USD']
```

Step 8: Split Dataset

```
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42)
```

Step 9: Train Models

```
linear_model = LinearRegression()
```

```
random_model = RandomForestRegressor()
```

```
linear_model.fit(X_train, y_train)
```

```
random_model.fit(X_train, y_train)
```

Step 10: Prediction

```
linear_pred = linear_model.predict(X_test)
```

```
random_pred = random_model.predict(X_test)
```

Step 11: Evaluation

```
print("\nLinear Regression Results")
```

```
print("MAE:", mean_absolute_error(y_test, linear_pred))
```

```
print("R2 Score:", r2_score(y_test, linear_pred))
```

```
print("\nRandom Forest Results")
```

```
print("MAE:", mean_absolute_error(y_test, random_pred))
```

```
print("R2 Score:", r2_score(y_test, random_pred))
```

Step 12: Predict Future Investment Trend

Example Input:

Year, Industry, City, Investment Type

```
sample_data = [[2026, 5, 3, 2]]
```

```
future_prediction = random_model.predict(sample_data)
```

```
print("\nPredicted Future Funding Amount:")
```

```
print(future_prediction[0])
```